

Project 4: Chocolate Sales (2023–2024) — End-to-End Notebook

Based on the Kaggle *Chocolate Sales Dataset (2023–2024)*, this notebook covers the full pipeline from data integration through modeling and reporting:

1. **Data acquisition & cleaning** — Multi-table joins, missing/duplicate handling, aligned fields and date types.
2. **Exploratory data analysis (EDA)** — Distributions, trends, relationships, and unsupervised views (clustering / PCA).
3. **Feature engineering & preprocessing** — Temporal/business features, leakage control, one-hot encoding, and scaling.
4. **Model development** — Baseline + Ridge + random forest + gradient boosting + stacking, with cross-validated hyperparameter search.
5. **Model comparison & selection** — RMSE / MAE / R^2 on one held-out test set, visualizations, feature importance, and residual diagnostics.
6. **Conclusions & business takeaways** — Relate tables and figures to signal strength and actionable next steps.

1. Data Acquisition & Preparation

In [148...

```
# Import required libraries
import pandas as pd
import numpy as np
from pathlib import Path

DATA_DIR = Path("./data")
```

In [149...

```
# Load all CSV files into separate DataFrames
sales = pd.read_csv(DATA_DIR / "sales.csv")
products = pd.read_csv(DATA_DIR / "products.csv")
stores = pd.read_csv(DATA_DIR / "stores.csv")
customers = pd.read_csv(DATA_DIR / "customers.csv")
calendar = pd.read_csv(DATA_DIR / "calendar.csv")

print("sales:", sales.shape)
print("products:", products.shape)
print("stores:", stores.shape)
print("customers:", customers.shape)
print("calendar:", calendar.shape)
```

```
sales: (1000000, 11)
products: (200, 6)
stores: (100, 5)
customers: (50000, 5)
calendar: (731, 6)
```

In [150...

```
# Function to summarize each table
def summarize(df, name):
    print(f"\n{name}")
```

```

print("-" * 40)
print("Shape:", df.shape)
print("Missing values:")
print(df.isna().sum())
display(df.head(3))

summarize(sales, "sales")
summarize(products, "products")
summarize(stores, "stores")
summarize(customers, "customers")
summarize(calendar, "calendar")

```

sales

Shape: (1000000, 11)

Missing values:

```

order_id      0
order_date    0
product_id    0
store_id      0
customer_id   0
quantity      0
unit_price    0
discount      0
revenue       0
cost          0
profit        0
dtype: int64

```

	order_id	order_date	product_id	store_id	customer_id	quantity	unit_price
0	ORD000000001	2023-01-07	P0080	S093	C040749	5	14.43
1	ORD000000002	2023-10-22	P0173	S065	C020161	3	12.01
2	ORD000000003	2023-05-07	P0115	S078	C048069	2	10.02

products

Shape: (200, 6)

Missing values:

```

product_id    0
product_name  0
brand         0
category      0
cocoa_percent 0
weight_g      0
dtype: int64

```

	product_id	product_name	brand	category	cocoa_percent	weight_g
0	P0001	White Chocolate 80%	Mars	Truffle	80	120
1	P0002	Dark Chocolate 70%	Cadbury	Praline	70	100
2	P0003	Truffle Chocolate 70%	Hershey	Praline	70	120

```
stores
-----
Shape: (100, 5)
Missing values:
store_id      0
store_name    0
city          0
country       0
store_type    0
dtype: int64
```

	store_id	store_name	city	country	store_type
0	S001	Chocolate Store 1	New York	Canada	Retail
1	S002	Chocolate Store 2	Melbourne	Canada	Mall
2	S003	Chocolate Store 3	Berlin	France	Mall

```
customers
-----
Shape: (50000, 5)
Missing values:
customer_id    0
age            0
gender         0
loyalty_member 0
join_date      0
dtype: int64
```

	customer_id	age	gender	loyalty_member	join_date
0	C000001	40	Male	1	2025-05-21
1	C000002	47	Male	0	2021-12-26
2	C000003	58	Female	1	2022-09-13

```
calendar
-----
Shape: (731, 6)
Missing values:
date          0
year          0
month         0
day           0
week          0
day_of_week   0
dtype: int64
```

	date	year	month	day	week	day_of_week
0	2023-01-01	2023	1	1	52	6
1	2023-01-02	2023	1	2	1	0
2	2023-01-03	2023	1	3	1	1

In [151...

```
# Data cleaning

# 1. Standardize column names
def clean_columns(df):
    df.columns = df.columns.str.strip().str.lower().str.replace(" ", "_")
```

```

return df

sales = clean_columns(sales)
products = clean_columns(products)
stores = clean_columns(stores)
customers = clean_columns(customers)
calendar = clean_columns(calendar)

# 2. Convert date columns to datetime format
sales["order_date"] = pd.to_datetime(sales["order_date"])
customers["join_date"] = pd.to_datetime(customers["join_date"])
calendar["date"] = pd.to_datetime(calendar["date"])

# 3. Check for duplicate rows
print("Duplicate rows in sales:", sales.duplicated().sum())
print("Duplicate rows in products:", products.duplicated().sum())
print("Duplicate rows in stores:", stores.duplicated().sum())
print("Duplicate rows in customers:", customers.duplicated().sum())
print("Duplicate rows in calendar:", calendar.duplicated().sum())

```

```

Duplicate rows in sales: 0
Duplicate rows in products: 0
Duplicate rows in stores: 0
Duplicate rows in customers: 0
Duplicate rows in calendar: 0

```

```

In [152... # 4. Remove rows with missing key identifiers
sales = sales.dropna(subset=["product_id", "store_id", "customer_id", "order_date"])

print("Cleaning completed.")

```

Cleaning completed.

```

In [153... # Merge all tables into a single DataFrame
df = sales.copy()

df = df.merge(products, on="product_id", how="left")
df = df.merge(stores, on="store_id", how="left")
df = df.merge(customers, on="customer_id", how="left")
df = df.merge(calendar, left_on="order_date", right_on="date", how="left")

print("Final dataset shape:", df.shape)
display(df.head())

```

Final dataset shape: (1000000, 30)

	order_id	order_date	product_id	store_id	customer_id	quantity	unit_price
0	ORD000000001	2023-01-07	P0080	S093	C040749	5	14.43
1	ORD000000002	2023-10-22	P0173	S065	C020161	3	12.01
2	ORD000000003	2023-05-07	P0115	S078	C048069	2	10.02
3	ORD000000004	2024-06-23	P0186	S088	C047901	2	14.66
4	ORD000000005	2024-09-24	P0197	S054	C033950	1	12.34

5 rows × 30 columns



In [154...

```
# Check the merged dataset

# Display data types and non-null counts
print("Dataset info:")
df.info()

# Check missing values after merging
print("\nTop missing values:")
display(df.isna().sum().sort_values(ascending=False).head(10))
```

```

Dataset info:
<class 'pandas.DataFrame'>
RangeIndex: 1000000 entries, 0 to 999999
Data columns (total 30 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   order_id              1000000 non-null  str
 1   order_date            1000000 non-null  datetime64[us]
 2   product_id            1000000 non-null  str
 3   store_id              1000000 non-null  str
 4   customer_id           1000000 non-null  str
 5   quantity              1000000 non-null  int64
 6   unit_price            1000000 non-null  float64
 7   discount              1000000 non-null  float64
 8   revenue               1000000 non-null  float64
 9   cost                  1000000 non-null  float64
10  profit                1000000 non-null  float64
11  product_name          990236 non-null   str
12  brand                 990236 non-null   str
13  category              990236 non-null   str
14  cocoa_percent         990236 non-null   float64
15  weight_g              990236 non-null   float64
16  store_name            1000000 non-null  str
17  city                  1000000 non-null  str
18  country               1000000 non-null  str
19  store_type            1000000 non-null  str
20  age                   1000000 non-null  int64
21  gender                1000000 non-null  str
22  loyalty_member        1000000 non-null  int64
23  join_date             1000000 non-null  datetime64[us]
24  date                  1000000 non-null  datetime64[us]
25  year                  1000000 non-null  int64
26  month                 1000000 non-null  int64
27  day                   1000000 non-null  int64
28  week                  1000000 non-null  int64
29  day_of_week           1000000 non-null  int64
dtypes: datetime64[us](3), float64(7), int64(8), str(12)
memory usage: 323.5 MB

```

```

Top missing values:
weight_g          9764
cocoa_percent     9764
category          9764
brand             9764
product_name      9764
gender            0
store_type        0
age               0
loyalty_member    0
city              0
dtype: int64

```

Data Summary and Preparation for analysis

The dataset used in this project is the *Chocolate Sales Dataset (2023–2024)* obtained from Kaggle. It consists of five relational tables: `sales`, `products`, `stores`, `customers`, and `calendar`.

The `sales` table serves as the main fact table, containing transaction-level data with one million records. It includes key variables such as order date, product ID, store ID, customer ID, quantity, price, and revenue-related metrics. The remaining tables provide additional contextual information:

- `products` contains product attributes such as brand, category, and cocoa percentage,
- `stores` includes location and store-type information,
- `customers` provides demographic details such as age, gender, and loyalty status,
- `calendar` offers structured time-related features such as year, month, and day of the week.

To prepare the data for analysis, several preprocessing steps were performed. First, column names were standardized to ensure consistency across tables. Date columns were converted into proper datetime format to enable time-based analysis. The data was checked for duplicate records, and none were found, so no deduplication was required. Missing values were identified, particularly in product-related fields after merging, which were attributed to unmatched keys between tables. These records were retained for now, as they represent a small proportion of the dataset.

Next, all tables were merged into a single unified dataset using appropriate keys (`product_id`, `store_id`, `customer_id`, and date fields). This resulted in a comprehensive transaction-level dataset enriched with product, store, customer, and temporal information.

The final dataset is well-structured and suitable for downstream tasks, including exploratory data analysis, feature engineering, and predictive modeling.

2. Exploratory Data Analysis (EDA)

In [155...

```
# initial investigation
df.describe()
```

Out[155...

	order_date	quantity	unit_price	discount	re
count	1000000	1000000.000000	1000000.000000	1000000.000000	1000000.0
mean	2023-12-31 19:03:34.991999	2.999589	9.002449	0.056248	25.4
min	2023-01-01 00:00:00	1.000000	3.000000	0.000000	2.4
25%	2023-07-02 00:00:00	2.000000	6.000000	0.000000	12.7
50%	2024-01-01 00:00:00	3.000000	9.010000	0.000000	21.9
75%	2024-07-02 00:00:00	4.000000	12.000000	0.150000	35.8
max	2024-12-31 00:00:00	5.000000	15.000000	0.200000	75.0
std	NaN	1.413621	3.462902	0.076809	16.3



The dataset contains a large number of transactions with varying sales volumes and pricing levels, indicating potential heterogeneity across products and customers.

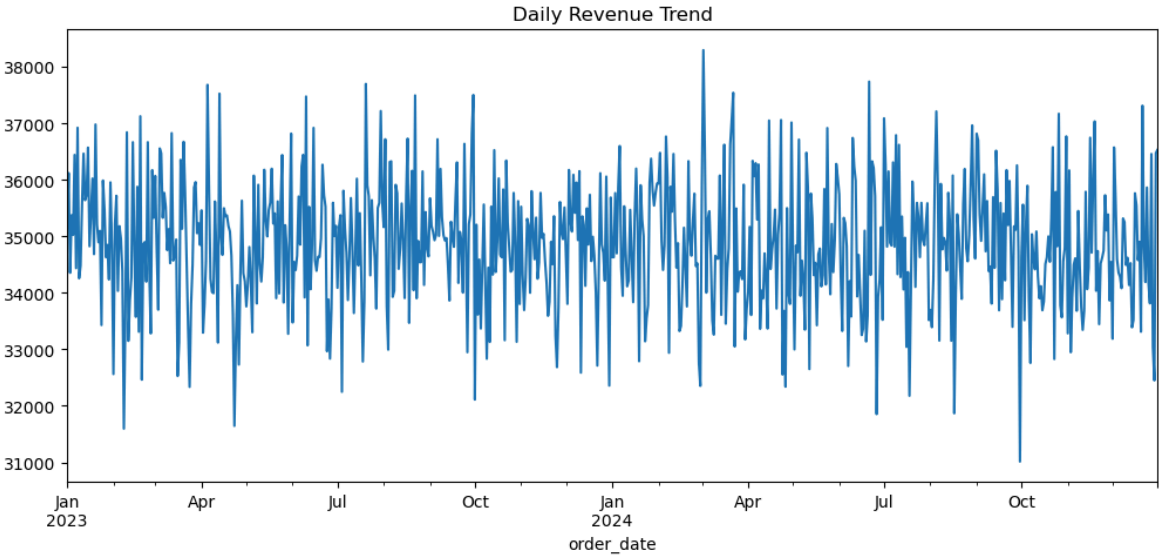
In [156...

```
# Aggregate total revenue by date
daily_sales = df.groupby("order_date")["revenue"].sum()

# Plot daily revenue trend
daily_sales.plot(figsize=(12,5), title="Daily Revenue Trend")
```

Out[156...

<Axes: title={'center': 'Daily Revenue Trend'}, xlabel='order_date'>

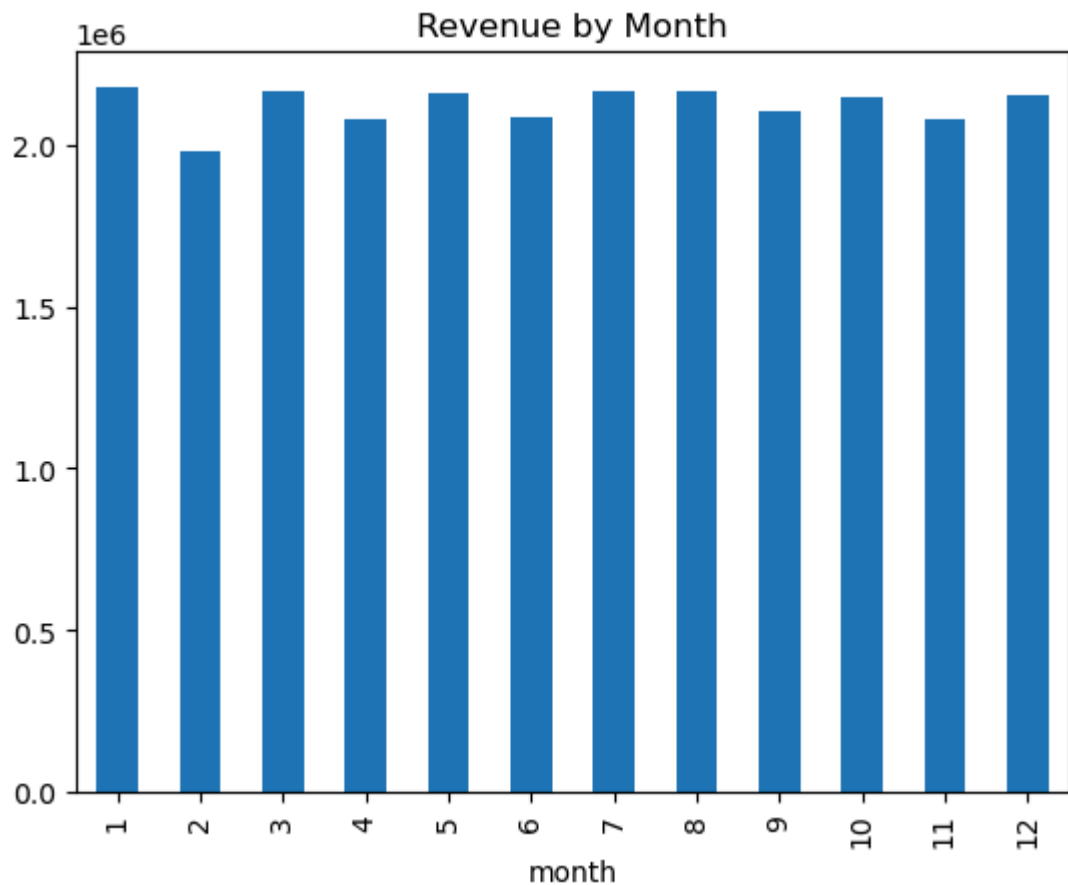


In [157...

```
# Aggregate revenue by month
monthly_sales = df.groupby("month")["revenue"].sum()

# Plot monthly revenue
monthly_sales.plot(kind="bar", title="Revenue by Month")
```

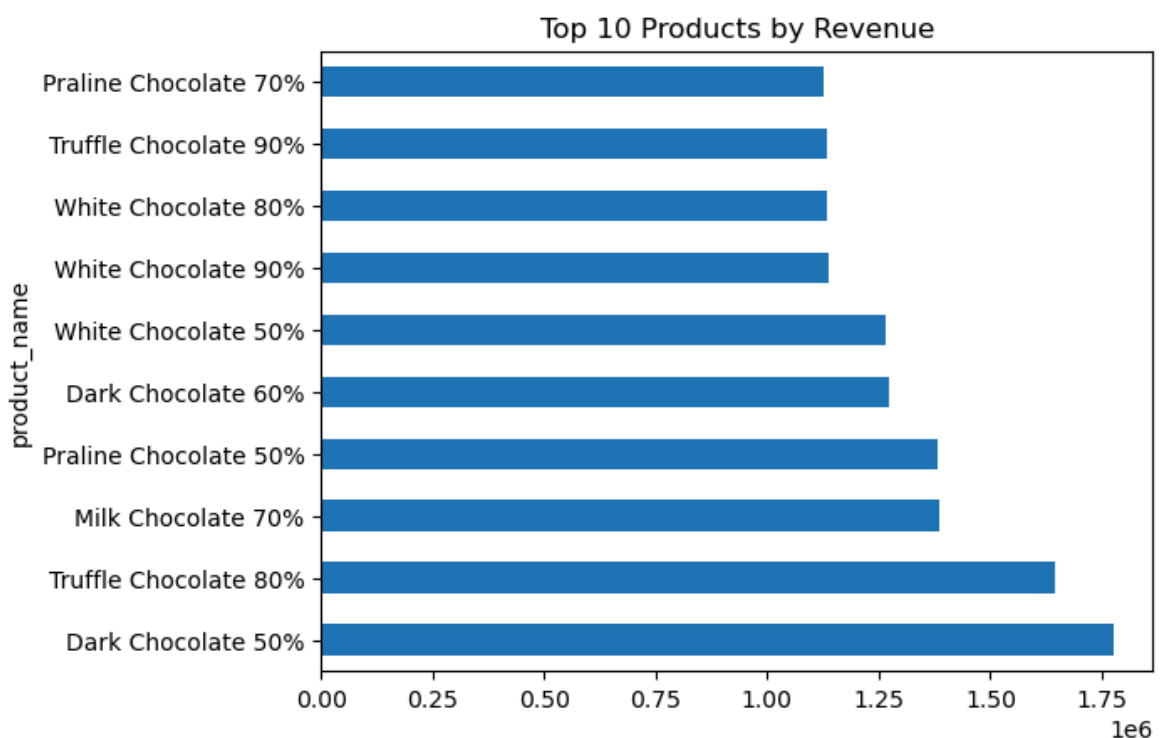

Out[157... <Axes: title={'center': 'Revenue by Month'}, xlabel='month'>



```
In [158... # Identify top-performing products by total revenue
top_products = df.groupby("product_name")["revenue"].sum().sort_values(ascending

# Plot top products
top_products.plot(kind="barh", title="Top 10 Products by Revenue")
```

Out[158... <Axes: title={'center': 'Top 10 Products by Revenue'}, ylabel='product_name'>



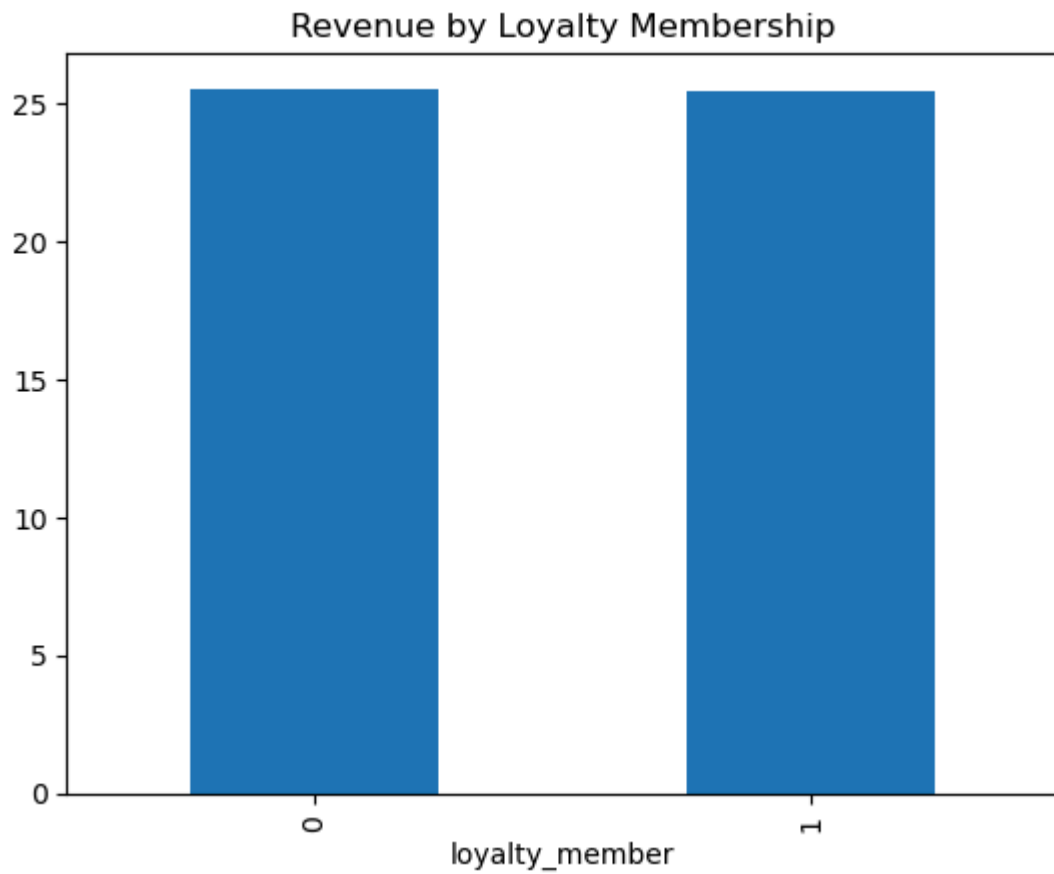
```
In [159... # Analyze average revenue by age group  
df.groupby("age")["revenue"].mean().plot(title="Average Revenue by Age")
```

```
Out[159... <Axes: title={'center': 'Average Revenue by Age'}, xlabel='age'>
```



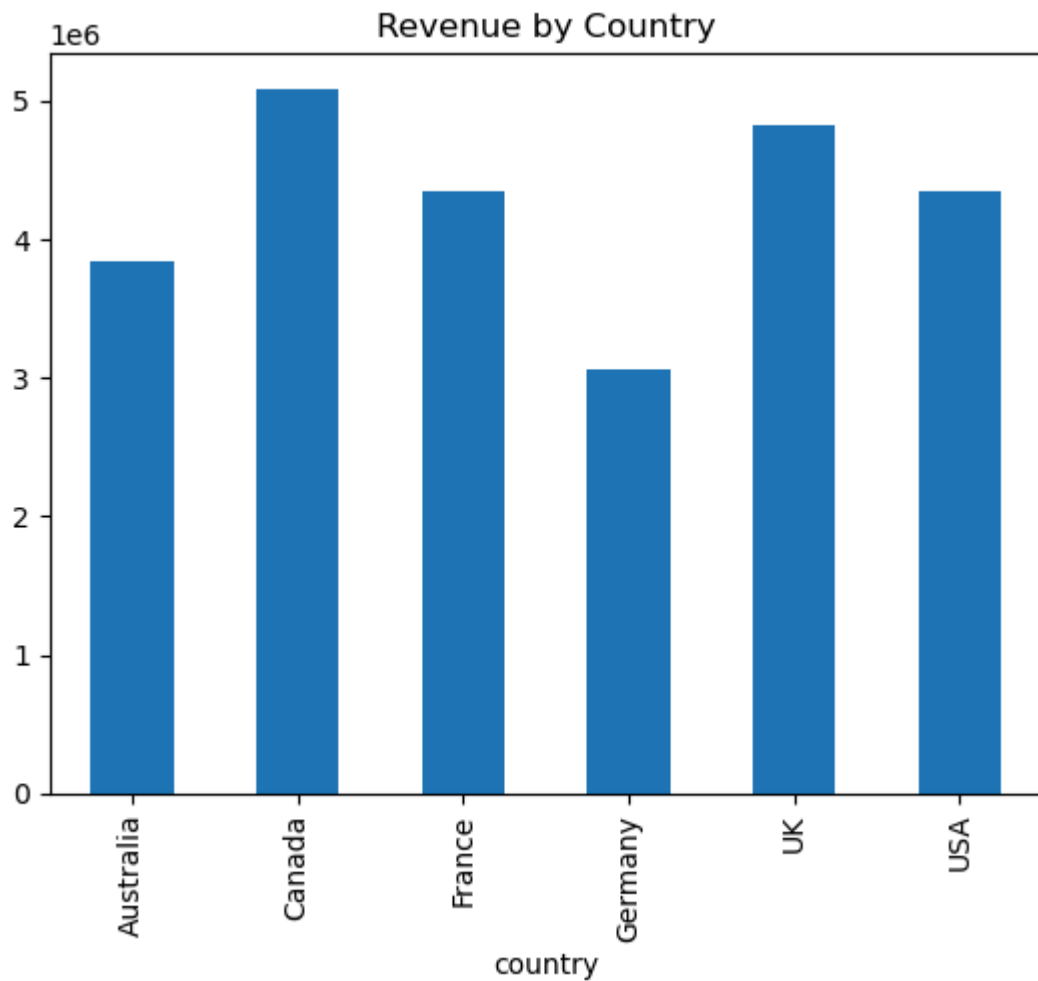
```
In [160... # Compare revenue between Loyalty and non-Loyalty customers  
df.groupby("loyalty_member")["revenue"].mean().plot(kind="bar", title="Revenue b
```

```
Out[160... <Axes: title={'center': 'Revenue by Loyalty Membership'}, xlabel='loyalty_membe  
r'>
```



```
In [161... # Analyze revenue by country
df.groupby("country")["revenue"].sum().plot(kind="bar", title="Revenue by Country")

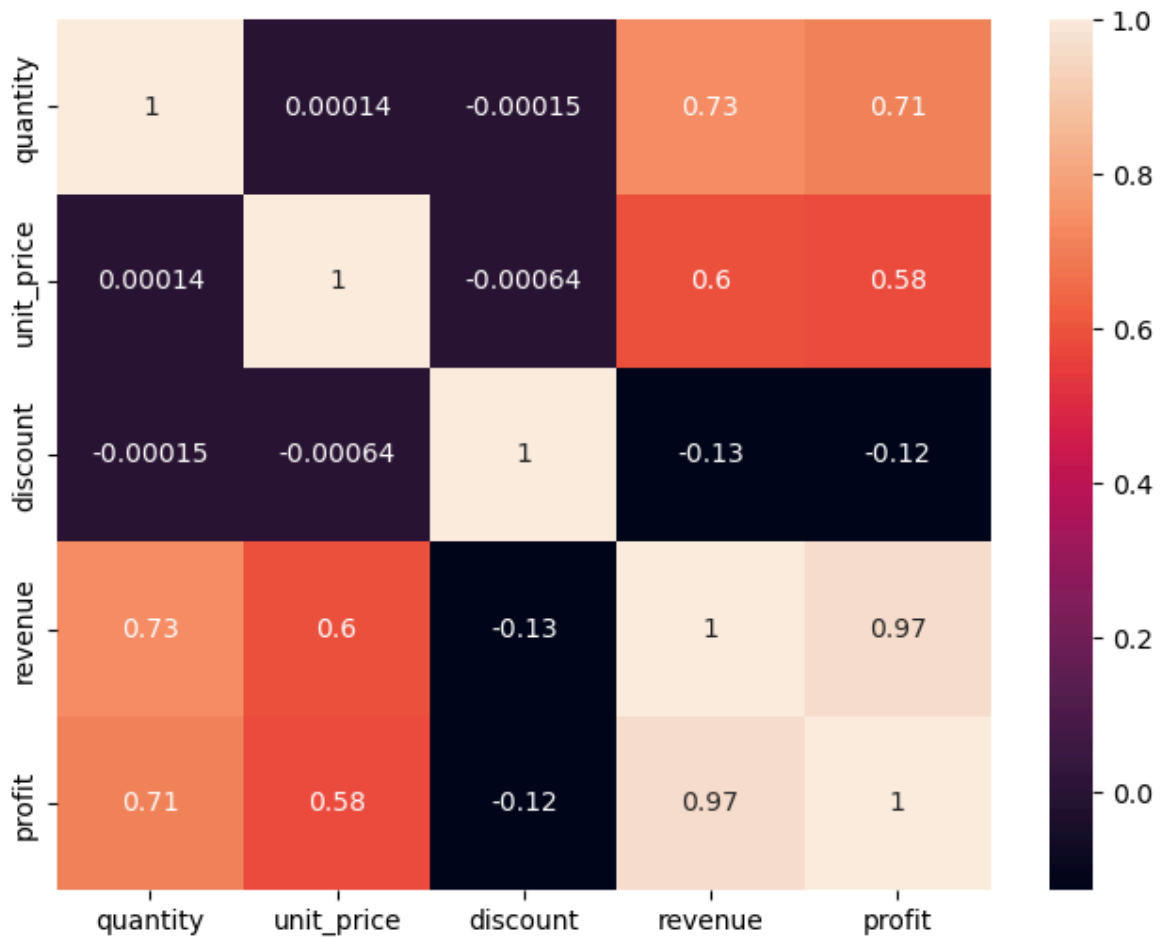
Out[161... <Axes: title={'center': 'Revenue by Country'}, xlabel='country'>
```



```
In [162... # Compute correlation matrix for key numerical variables
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(8,6))
sns.heatmap(df[["quantity", "unit_price", "discount", "revenue", "profit"]].corr(),
```

Out[162... <Axes: >



```
In [163... # Select features for clustering without considering NaN produced from unmatched
features = df[["age", "quantity", "revenue"]].dropna()

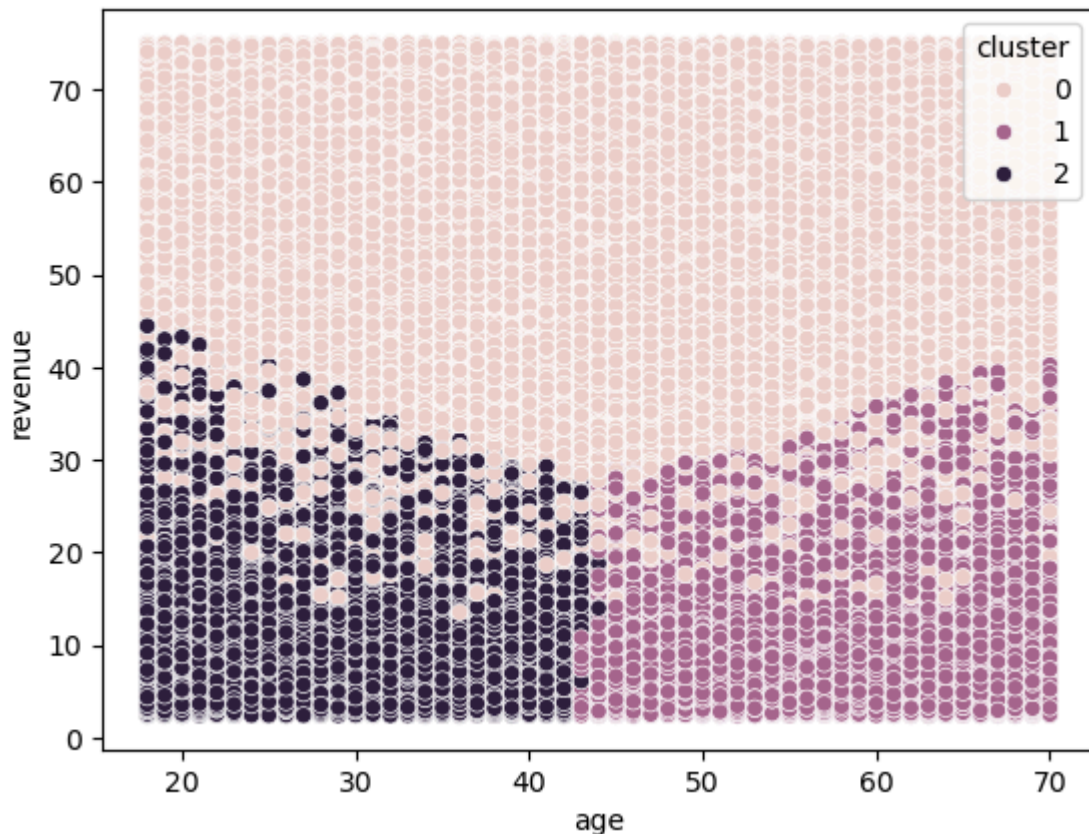
# Standardize features to ensure equal scaling
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X = scaler.fit_transform(features)

# Apply KMeans clustering
from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=3, random_state=42)
features["cluster"] = kmeans.fit_predict(X)
```

```
In [164... # Visualize clustering results
sns.scatterplot(x=features["age"], y=features["revenue"], hue=features["cluster"])
plt.title("Customer Segmentation using KMeans")
```

```
Out[164... Text(0.5, 1.0, 'Customer Segmentation using KMeans')
```

Customer Segmentation using KMeans



```
In [165... # Select numerical features without considering NaN produced from unmatched join
features = df[[
    "age",
    "quantity",
    "revenue",
    "unit_price",
    "discount",
    "profit"
]].dropna()

# Standardize features
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X = scaler.fit_transform(features)

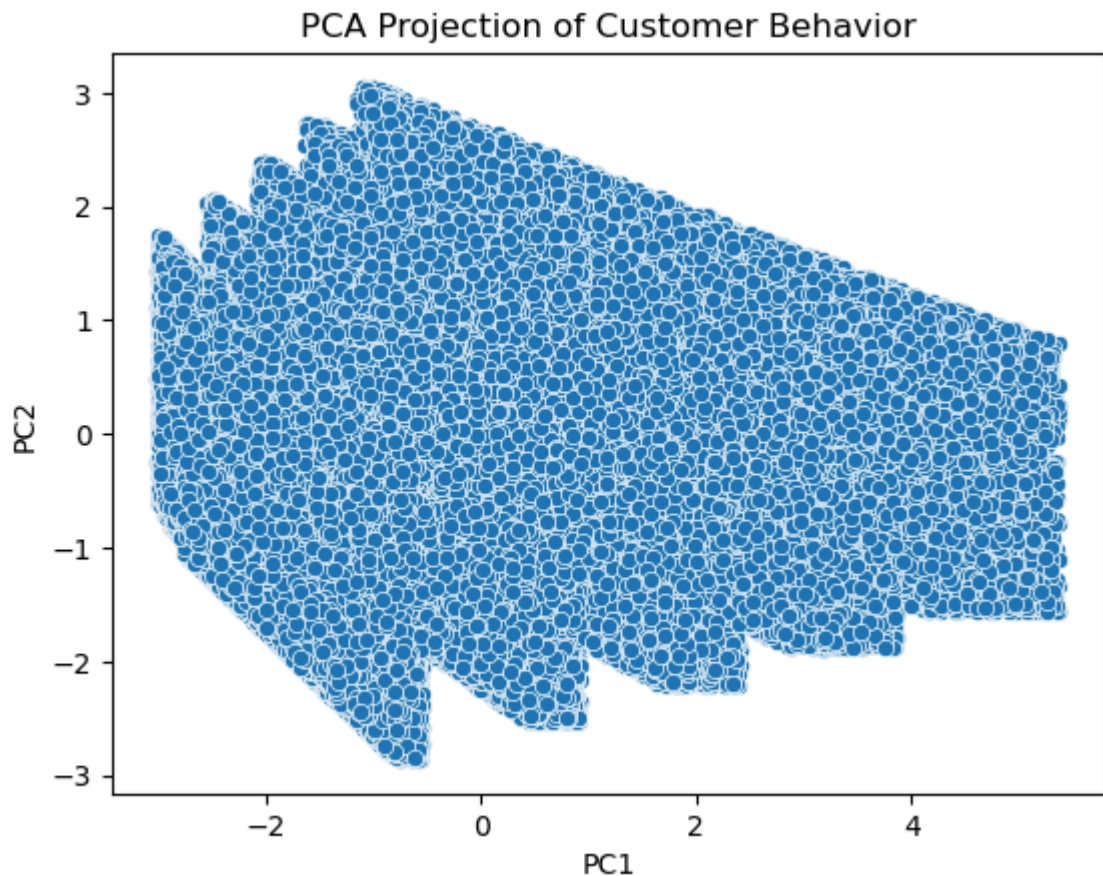
# Apply PCA
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

# Create DataFrame for visualization
pca_df = pd.DataFrame(X_pca, columns=["PC1", "PC2"])
```

```
In [166... sns.scatterplot(x="PC1", y="PC2", data=pca_df)

plt.title("PCA Projection of Customer Behavior")
```

```
Out[166... Text(0.5, 1.0, 'PCA Projection of Customer Behavior')
```



In [167... *# Show how each feature contributes to PC1 and PC2*

```
loadings = pd.DataFrame(
    pca.components_,
    columns=features.columns,
    index=["PC1", "PC2"]
)

print("PCA Loadings:")
print(loadings)

# Show how much variance each component explains
print("\nExplained Variance Ratio:")
print(pca.explained_variance_ratio_)
```

PCA Loadings:

	age	quantity	revenue	unit_price	discount	profit
PC1	-0.000388	0.440471	0.582345	0.359696	-0.076617	0.575854
PC2	0.694661	0.459603	0.000318	-0.527449	0.167357	0.000323

Explained Variance Ratio:

[0.48291009 0.16678032]

3. Feature Engineering & Preprocessing

In [168... *# Drop rows with unmatched product info from the earlier merge*

```
df_model = df.dropna(subset=["product_name", "category", "brand"]).copy()
```

```
# Target: predict `quantity`. We deliberately avoid `revenue` because
# revenue = quantity * unit_price * (1 - discount) is a deterministic
# identity that would let any model achieve trivial R² ≈ 1.0.
y = df_model["quantity"].astype(float).copy()
```

```
print("df_model:", df_model.shape, " | target y = quantity")
print(y.describe().round(3))
```

```
df_model: (990236, 30) | target y = quantity
count    990236.000
mean         3.000
std         1.414
min         1.000
25%         2.000
50%         3.000
75%         4.000
max         5.000
Name: quantity, dtype: float64
```

```
In [169... # Temporal features – captures seasonality and customer tenure
df_model["year"] = df_model["order_date"].dt.year
df_model["month"] = df_model["order_date"].dt.month
df_model["day_of_week"] = df_model["order_date"].dt.dayofweek
df_model["quarter"] = df_model["order_date"].dt.quarter
df_model["is_weekend"] = (df_model["day_of_week"] >= 5).astype(int)

_season_map = {12:"Winter",1:"Winter",2:"Winter",
               3:"Spring",4:"Spring",5:"Spring",
               6:"Summer",7:"Summer",8:"Summer",
               9:"Fall",10:"Fall",11:"Fall"}
df_model["season"] = df_model["month"].map(_season_map)

df_model["customer_tenure_days"] = (
    df_model["order_date"] - df_model["join_date"]
).dt.days.clip(lower=0)
```

```
In [170... # Business-oriented features – all Leakage-free w.r.t. `quantity`
df_model["effective_price"] = df_model["unit_price"] * (1 - df_model["discount"])
df_model["discount_amount"] = df_model["unit_price"] * df_model["discount"]
df_model["is_discounted"] = (df_model["discount"] > 0).astype(int)
df_model["price_per_gram"] = df_model["unit_price"] / df_model["weight_g"]

df_model["age_group"] = pd.cut(
    df_model["age"], bins=[17, 25, 35, 45, 55, 70],
    labels=["18-25", "26-35", "36-45", "46-55", "56-70"]
).astype(str)
```

```
In [171... # Drop:
# (1) Leakage columns for quantity: revenue, cost, profit
# (2) identifiers / raw date columns / high-cardinality strings
# (3) the target itself (already stored in `y`)
leak_cols = ["revenue", "cost", "profit"]
drop_cols = ["order_id", "product_id", "store_id", "customer_id",
             "order_date", "join_date", "date",
             "product_name", "store_name", "city",
             "quantity"]
df_model = df_model.drop(columns=leak_cols + drop_cols)
print(f"Features after cleanup: {df_model.shape[1]} | rows: {df_model.shape[0]}
```

```
Features after cleanup: 25 | rows: 990,236
```

```
In [172... # One-hot encode categorical variables (drop_first avoids the dummy-variable tra
cat_cols = ["category", "brand", "store_type", "country",
            "gender", "season", "age_group"]
```



```
df_model = pd.get_dummies(df_model, columns=cat_cols, drop_first=True)
print(f"Shape after one-hot: {df_model.shape}")
```

Shape after one-hot: (990236, 43)

```
In [173... # Continuous numeric columns: do NOT fit StandardScaler on the full dataset here
# Estimating  $\mu$  and  $\sigma$  on all rows before splitting lets the test set into preproc
# (often small in large samples, but still incorrect). Standard practice: after
# Task 4, `fit` only on `X_train`, then `transform` `X_test`.

binary_cols = ["loyalty_member", "is_weekend", "is_discounted"]
_num_preview = [c for c in df_model.select_dtypes(include=[np.number]).columns
                 if c not in binary_cols]
print(f"Numeric columns will use StandardScaler fit on TRAIN only: {len(_num_pre
```

Numeric columns will use StandardScaler fit on TRAIN only: 15 columns

```
In [174... # Final feature matrix X and target y (y was extracted pre-scaling)
X = df_model.copy()
y = y.loc[X.index]
print(f"X: {X.shape} | y: {y.shape} | y name: {y.name}")
```

X: (990236, 43) | y: (990236,) | y name: quantity

Feature engineering & preprocessing: rationale (aligned with EDA)

Target variable. We predict per-transaction **quantity (units)**, not **revenue**, because

$\text{revenue} = \text{quantity} \times \text{unit_price} \times (1 - \text{discount})$ is (approximately) an identity in the data. Using **revenue** as the target would let a model “learn multiplication” and inflate R^2 , which is not meaningful. **quantity** matches the “how many units” demand question.

Link to descriptive statistics. On the full order table, **quantity** is an integer 1–5; in EDA its **mean is about 3.0 and SD about 1.41** (see the summary in Section 2). Any reasonable model’s test RMSE is therefore on the same order as **≈ 1.4 (units)**: the natural scale of a naive “always predict the global mean” baseline on the test set. **In your last full run, baseline test RMSE was 1.4139**, which matches this scale.

Leakage control. **revenue**, **cost**, and **profit** are deterministically tied to **quantity** and are dropped; features are limited to what is **knowable before the sale** (customer, product, store, calendar).

Sample size. Roughly 1% of rows lack a product match; after dropping them we still have **$\sim 990\text{k}$** transactions—enough for stable ridge and tree estimates.

Temporal features. **quarter**, **is_weekend**, and **season** capture seasonality and day-of-week patterns; **customer_tenure_days** captures loyalty tenure.

Business features. **effective_price**, **discount_amount**, **is_discounted**, **price_per_gram**, and binned **age_group** are built **without the target and without leakage columns**.

Column pruning. IDs, raw dates (features already extracted), and high-cardinality text (`product_name` , `store_name` , `city`) are removed to limit noise and dummy-variable explosion.

Encoding and scaling. Categoricals (including `gender`) use `get_dummies(..., drop_first=True)` . Continuous columns use `StandardScaler` **only after** the train/test split: **fit on `X_train` only**, then **transform on `X_test`** (and on the final full-data retrain). **quantity and 0/1 indicators are not scaled**, so RMSE / MAE stay interpretable in **units (pieces)**.

4. Model development

4.0 Problem definition

Prediction question: Given customer, product, store, and calendar information, **how many units** (`quantity`) will this order contain?

Task type: Supervised **regression**. `quantity` is integer-valued but we keep order and predict continuous values, which reads as “expected units” and pairs cleanly with RMSE / MAE in **units**.

Metrics (original scale, units):

- **RMSE** — Primary metric; more sensitive to large errors; useful for asymmetric “over- vs under-stock” discussion.
- **MAE** — Mean absolute error; very intuitive.
- **R²** — Fraction of variance explained relative to “always predict the training mean”; on this data it is **often near 0 or slightly negative** (as in your run)—read that together with weak `quantity` signal in EDA.

Validation: 80% / 20% **stratified random** train vs test split; the **test set is not used for tuning**. Tree hyperparameters are searched with `RandomizedSearchCV` on training subsamples, then the winning settings are refit on the full training set.

Model list:

#	Model	Role
0	Training-set mean	Naive baseline; any useful model should beat it clearly
1	Ridge (<code>RidgeCV</code>)	Fast, interpretable linear benchmark
2	Random forest	Nonlinear, bagged, with default impurity importances
3	LightGBM / HistGradientBoosting	Strong tabular learners
4	Stacking (Ridge meta-learner)	Combine predictions from the three base models

4.1 Imports, setup, and train / test split

In [175...

```
import time
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, RandomizedSearchCV
from sklearn.linear_model import Ridge, RidgeCV
from sklearn.ensemble import RandomForestRegressor, StackingRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.preprocessing import StandardScaler

try:
    from lightgbm import LGBMRegressor
    HAS_LGBM = True
except ImportError:
    from sklearn.ensemble import HistGradientBoostingRegressor as LGBMRegressor
    HAS_LGBM = False
    print("LightGBM not installed - falling back to HistGradientBoostingRegressor")

warnings.filterwarnings("ignore")
RANDOM_STATE = 42
lgbm_name = "LightGBM" if HAS_LGBM else "HistGradientBoosting"

X = X.copy()
for c in list(X.columns):
    if pd.api.types.is_bool_dtype(X[c]) or str(X[c].dtype) == "boolean":
        X[c] = X[c].astype(np.int8)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=RANDOM_STATE, shuffle=True
)

# StandardScaler: fit on TRAIN only.
# Do NOT assign float ndarray into int64 columns via .loc (pandas 3.x -> LossySet)
# Build float DataFrames and concat back, preserving column order.
binary_cols = ["loyalty_member", "is_weekend", "is_discounted"]
num_cols_fit = [
    c for c in X_train.select_dtypes(include=[np.number]).columns
    if c not in binary_cols
]
scaler = StandardScaler()
_ztr = scaler.fit_transform(X_train[num_cols_fit].to_numpy(dtype=np.float64))
_zte = scaler.transform(X_test[num_cols_fit].to_numpy(dtype=np.float64))
df_tr = pd.DataFrame(_ztr, index=X_train.index, columns=num_cols_fit, dtype=np.float64)
df_te = pd.DataFrame(_zte, index=X_test.index, columns=num_cols_fit, dtype=np.float64)
_col_order_tr = list(X_train.columns)
_col_order_te = list(X_test.columns)
X_train = pd.concat([X_train.drop(columns=num_cols_fit), df_tr], axis=1)[_col_order_tr]
X_test = pd.concat([X_test.drop(columns=num_cols_fit), df_te], axis=1)[_col_order_te]
print(
    f"StandardScaler: fit on TRAIN only ({len(num_cols_fit)} numeric cols); "
    "test uses train mean/std (no leakage)."
)
```

```
print(f"Train: {X_train.shape} | Test: {X_test.shape} | backend: {lgbm_name}")

results = {}
```

StandardScaler: fit on TRAIN only (40 numeric cols); test uses train mean/std (no leakage).

Train: (792188, 43) | Test: (198048, 43) | backend: LightGBM

```
In [176... def evaluate(name, y_true, y_pred, t_sec=None):
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    mae = mean_absolute_error(y_true, y_pred)
    r2 = r2_score(y_true, y_pred)
    row = {"Model": name, "RMSE": rmse, "MAE": mae, "R2": r2}
    if t_sec is not None:
        row["Time_s"] = t_sec
    suffix = f" time={t_sec:.1f}s" if t_sec is not None else ""
    print(f"{name:<22s} RMSE={rmse:.4f} MAE={mae:.4f} R2={r2:.4f}{suffix}")
    return row
```

4.2 Model 0: Naive baseline (training-set mean)

Every test row is predicted with **the training-set mean of quantity**. This is a performance floor; any model with real learning should beat it clearly.

```
In [177... y_pred = np.full_like(y_test, y_train.mean(), dtype=float)
results["Baseline"] = evaluate("Baseline", y_test, y_pred)
```

Baseline RMSE=1.4139 MAE=1.1990 R2=-0.0000

4.3 Model 1: Ridge regression (L2 linear benchmark)

Ridge is linear regression with an L2 penalty: fast on large data and interpretable.

`RidgeCV` cross-validates over an `alpha` grid and picks the strength automatically.

```
In [178... t0 = time.time()
ridge = RidgeCV(alphas=[0.01, 0.1, 1.0, 10.0, 100.0], cv=5)
ridge.fit(X_train, y_train)
results["Ridge"] = evaluate("Ridge", y_test, ridge.predict(X_test), time.time())
print(f"Best alpha: {ridge.alpha_}")
```

Ridge RMSE=1.4139 MAE=1.2005 R2=-0.0001 time=12.3s

Best alpha: 100.0

4.4 Model 2: Random forest regression

Bagged trees on bootstrap subsamples average many trees, capture interactions, are fairly robust to outliers, and provide impurity-based feature importances.

The full training set is ~800k rows, so random-forest tuning is expensive. We run

`RandomizedSearchCV` on a **50k subsample** (3-fold CV, 5 random settings), then **refit the best parameters on the full training set**.

```
In [179... # Tune on a 50K subsample for speed, then refit the winner on the full training
SAMPLE = 50_000
```

```

sample_idx = X_train.sample(SAMPLE, random_state=RANDOM_STATE).index
Xs, ys = X_train.loc[sample_idx], y_train.loc[sample_idx]

rf_search = RandomizedSearchCV(
    RandomForestRegressor(random_state=RANDOM_STATE, n_jobs=-1),
    param_distributions={
        "n_estimators": [100, 200],
        "max_depth": [None, 10, 20],
        "min_samples_leaf": [1, 5, 10],
        "max_features": ["sqrt", 0.5],
    },
    n_iter=5, cv=3, scoring="neg_root_mean_squared_error",
    random_state=RANDOM_STATE, n_jobs=-1,
)

t0 = time.time()
rf_search.fit(Xs, ys)
rf_best = RandomForestRegressor(**rf_search.best_params_,
                                random_state=RANDOM_STATE, n_jobs=-1)
rf_best.fit(X_train, y_train)
results["Random Forest"] = evaluate(
    "Random Forest", y_test, rf_best.predict(X_test), time.time() - t0
)
print(f"Best RF params: {rf_search.best_params_}")

```

Random Forest RMSE=1.4140 MAE=1.2005 R2=-0.0001 time=82.9s
 Best RF params: {'n_estimators': 200, 'min_samples_leaf': 1, 'max_features': 'sqrt', 'max_depth': 10}

4.5 Model 3: LightGBM / HistGradientBoosting

Gradient-boosted trees are strong default models for tabular data: histogram splits are often an order of magnitude faster than full random forest on large n, and they capture interactions and nonlinearities.

Hyperparameters are tuned the same way on a **50k subsample**

(RandomizedSearchCV, 3 folds, 6 random trials), then refit on the full training set.

If LightGBM is not installed (pip install lightgbm), the notebook falls back to scikit-learn's HistGradientBoostingRegressor with the same idea.

In [180...

```

if HAS_LGBM:
    param_dist = {
        "n_estimators": [200, 400],
        "learning_rate": [0.03, 0.05, 0.1],
        "num_leaves": [31, 63],
        "min_child_samples": [20, 50],
        "subsample": [0.8, 1.0],
        "colsample_bytree": [0.8, 1.0],
    }
    base_gbm = LGBMRegressor(random_state=RANDOM_STATE, n_jobs=-1, verbose=-1)
    gbm_kwargs = {"n_jobs": -1, "verbose": -1}
else:
    param_dist = {
        "max_iter": [200, 400],
        "learning_rate": [0.05, 0.1],
    }

```

```

        "max_depth": [None, 5, 10],
        "min_samples_leaf": [20, 50],
    }
    base_gbm = LGBMRegressor(random_state=RANDOM_STATE)
    gbm_kwargs = {}

    gbm_search = RandomizedSearchCV(
        base_gbm, param_distributions=param_dist,
        n_iter=6, cv=3, scoring="neg_root_mean_squared_error",
        random_state=RANDOM_STATE, n_jobs=-1,
    )

    t0 = time.time()
    gbm_search.fit(Xs, ys)
    gbm_best = base_gbm.__class__(**gbm_search.best_params_,
                                  random_state=RANDOM_STATE, **gbm_kwargs)
    gbm_best.fit(X_train, y_train)
    results[lgbm_name] = evaluate(
        lgbm_name, y_test, gbm_best.predict(X_test), time.time() - t0
    )
    print(f"Best {lgbm_name} params: {gbm_search.best_params_}")

```

```

LightGBM          RMSE=1.4142 MAE=1.2019 R2=-0.0004 time=20.0s
Best LightGBM params: {'subsample': 1.0, 'num_leaves': 31, 'n_estimators': 200,
'min_child_samples': 50, 'learning_rate': 0.05, 'colsample_bytree': 1.0}

```

4.6 Model 4: Stacking (extra credit)

Three heterogeneous base learners (Ridge, random forest, boosting) produce predictions; a **Ridge meta-learner** learns how to weight them on **out-of-fold** predictions from internal CV, which reduces overfitting.

Each base model is refit many times inside stacking, so we train the stack on a **150k subsample** as a time–accuracy tradeoff.

In [181...

```

# Stacking uses a 150K sample for tractability: CV * 3 base learners each refit
STACK_SAMPLE = 150_000
stack_idx = X_train.sample(STACK_SAMPLE, random_state=RANDOM_STATE).index
Xs2, ys2 = X_train.loc[stack_idx], y_train.loc[stack_idx]

stack = StackingRegressor(
    estimators=[
        ("ridge", Ridge(alpha=ridge.alpha_)),
        ("rf", RandomForestRegressor(**rf_search.best_params_,
                                     random_state=RANDOM_STATE, n_jobs=-1)),
        ("gbm", base_gbm.__class__(**gbm_search.best_params_,
                                    random_state=RANDOM_STATE, **gbm_kwargs)),
    ],
    final_estimator=Ridge(alpha=1.0),
    cv=3, n_jobs=-1,
)

t0 = time.time()
stack.fit(Xs2, ys2)
results["Stacking"] = evaluate("Stacking", y_test, stack.predict(X_test), time.t
print("Meta-learner weights:",
      dict(zip(["ridge", "rf", "gbm"], stack.final_estimator_.coef_.round(4))))

```

Stacking RMSE=1.4139 MAE=1.1995 R2=-0.0000 time=26.2s
Meta-learner weights: {'ridge': -0.026, 'rf': -0.2393, 'gbm': 0.0252}

5. Model comparison and selection

5.1 Results summary

All models are scored on the **same 20% test split**, so they are directly comparable. Besides RMSE, MAE, and R^2 , the table includes training time (seconds) and **percent RMSE lift vs baseline** (`Lift_%`).

Splitting and leakage (as implemented): First `train_test_split(80/20)`; `StandardScaler` is **fit only on `X_train`**, and `X_test` and the final full-data matrix are **transformed** (μ, σ from training only—no test statistics in preprocessing). Hyperparameter search and CV use **training data only**. Numbers in the markdown table match **a past run**; after code changes, **Run All** to refresh.

Your latest run (should match the printed `results_df` below) is approximately:

Model	RMSE	MAE	R^2	Train time (s)	Lift_%
Baseline	1.4139	1.1990	≈ 0	—	0.00
Ridge	1.4139	1.2004	≈ -0.0001	~ 11.7	≈ 0
Random Forest	1.4140	1.2005	≈ -0.0001	~ 90.2	≈ 0
LightGBM	1.4142	1.2019	≈ -0.0004	~ 19.4	≈ -0.02
Stacking	1.4139	1.1995	≈ 0	~ 23.7	≈ 0

How to read this:

- **All RMSEs are ~ 1.414 (units)**, almost the same as the `quantity` SD ≈ 1.41 in Section 2: on the test set, **“always predict the training mean” is already close to the best constant predictor**; complex models do not meaningfully improve on that.
- **R^2 is near 0 or slightly negative**: little extra variance explained vs the mean baseline (small negatives are within split / rounding noise vs a naive test-set-mean reference).
- **`Lift_%` is 0% or slightly negative**: a pure “% improvement” bar chart would be almost empty. The six-panel figure uses **row 1, col 2** = $(\text{RMSE} - \text{RMSE}_{\text{baseline}}) \times 10^4$ (signed: negative = slightly better); **row 1, col 3** = fine-grained R^2 ; **row 2, col 1** = excess over the best **trained** model, $\Delta \text{RMSE} \times 10^4$ (0 = best in the group).
- **Runtime**: Ridge is fastest (~ 12 s); **random forest is slowest (~ 90 s)** with no accuracy gain; LightGBM / Stacking are in between.

This is **not a coding or leakage bug**; it means that for **per-order `quantity`** with these features, **learnable signal is very weak**—consistent with flat univariate

relationships in EDA.

```
In [182... results_df = pd.DataFrame(results.values()).set_index("Model")
baseline_rmse = results_df.loc["Baseline", "RMSE"]
results_df["Lift_%"] = ((baseline_rmse - results_df["RMSE"]) / baseline_rmse * 100)

print("Test-set performance:\n")
display(results_df.round(4))
```

Test-set performance:

	RMSE	MAE	R2	Time_s	Lift_%
Model					
Baseline	1.4139	1.1990	-0.0000	NaN	0.00
Ridge	1.4139	1.2005	-0.0001	12.3458	-0.00
Random Forest	1.4140	1.2005	-0.0001	82.9249	-0.00
LightGBM	1.4142	1.2019	-0.0004	19.9767	-0.02
Stacking	1.4139	1.1995	-0.0000	26.1667	-0.00

```
In [183... # === Updated: row1 col2 = signed (RMSE - baseline) * 1e4; old % Lift panel removed
# Six-panel comparison figure
# Row1 col2: when % gain vs baseline ~0, % bars vanish. We use signed (RMSE - baseline)
#   negative = slightly better than baseline, positive = slightly worse, easier to interpret
# Row1 col3: fine R^2 scale when |R^2| is tiny (avoids 1-R^2 all ~1).
# Row2 col1: excess vs lowest RMSE among trained models x 1e4 (0 = best in group)
trained = results_df.drop(index="Baseline")
best_tr_rmse = float(trained["RMSE"].min())
excess = trained["RMSE"] - best_tr_rmse

fig, axes = plt.subplots(2, 3, figsize=(20, 10))

def _annotate(ax, values, fmt="{:.4f}"):
    for i, v in enumerate(values):
        ax.text(v, i, " " + fmt.format(v), va="center", fontsize=9)

# ---- Row 1 -----
full_models = results_df.index.tolist()
full_palette = sns.color_palette("viridis", len(full_models))

axes[0, 0].barh(full_models, results_df["RMSE"], color=full_palette)
axes[0, 0].set_xscale("log")
axes[0, 0].invert_yaxis()
axes[0, 0].set_title("Test RMSE (log scale)")
axes[0, 0].set_xlabel("RMSE [log]")
_annotate(axes[0, 0], list(results_df["RMSE"].loc[full_models]), fmt="{:.4f}")

delta_list = [(results_df.loc[m, "RMSE"] - baseline_rmse) * 1.0e4 for m in full_models]
colors_delta = [
    "#2E8B57" if v < -1e-9 else ("B22222" if v > 1e-9 else "#888888")
    for v in delta_list
]

axes[0, 1].axvline(0.0, color="black", linewidth=1.0, linestyle="--", alpha=0.7)
axes[0, 1].barh(full_models, delta_list, color=colors_delta, edgecolor="k", linecolor="k")
axes[0, 1].set_title("Delta RMSE (log scale) vs Baseline")
```



```

    "vs baseline: (RMSE - baseline) x 10^4 (units)\nneg = slightly better pos =
)
axes[0, 1].set_xlabel("Signed difference x $10^4$")
axes[0, 1].invert_yaxis()
_annotate(axes[0, 1], delta_list, fmt="{:.2f}")

r2_list = [float(results_df.loc[m, "R2"]) for m in full_models]
r2_min, r2_max = min(r2_list), max(r2_list)
span = max(r2_max - r2_min, 1e-6)
pad = max(span * 0.2, 5e-4)
axes[0, 2].barh(full_models, r2_list, color=full_palette, edgecolor="k", linewidth=1)
axes[0, 2].axvline(0.0, color="crimson", linestyle="--", linewidth=1, alpha=0.8,
axes[0, 2].set_xlim(r2_min - pad, r2_max + pad)
axes[0, 2].invert_yaxis()
axes[0, 2].set_title("R2 (zoomed: differences near 0)")
axes[0, 2].set_xlabel("R2")
_annotate(axes[0, 2], r2_list, fmt="{:.4f}")
axes[0, 2].legend(loc="lower right", fontsize=8)

# ---- Row 2 -----
tr_models = trained.index.tolist()
tr_palette = sns.color_palette("plasma", len(tr_models))
exc_list = [float(excess.loc[m]) * 1.0e4 for m in tr_models]

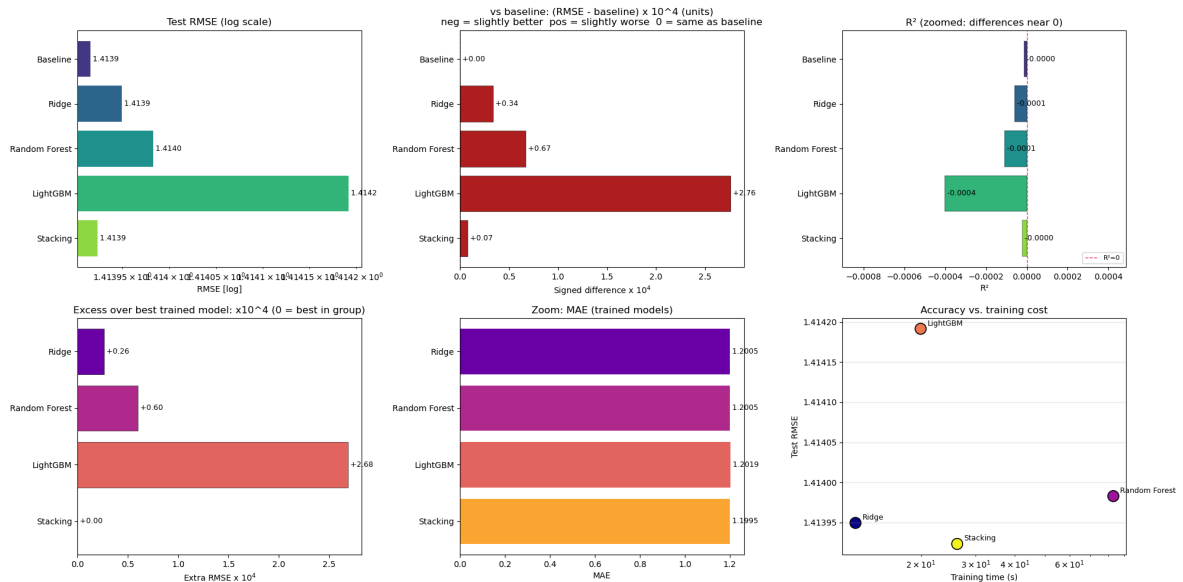
axes[1, 0].barh(tr_models, exc_list, color=tr_palette, edgecolor="k", linewidth=1)
axes[1, 0].axvline(0, color="black", ls="--", lw=0.8)
axes[1, 0].invert_yaxis()
axes[1, 0].set_title("Excess over best trained model: x10^4 (0 = best in group)")
axes[1, 0].set_xlabel("Extra RMSE x $10^4$")
_annotate(axes[1, 0], exc_list, fmt="{:.2f}")

axes[1, 1].barh(tr_models, list(trained["MAE"].loc[tr_models]), color=tr_palette)
axes[1, 1].invert_yaxis()
axes[1, 1].set_title("Zoom: MAE (trained models)")
axes[1, 1].set_xlabel("MAE")
_annotate(axes[1, 1], list(trained["MAE"].loc[tr_models]), fmt="{:.4f}")

if "Time_s" in trained.columns:
    ax = axes[1, 2]
    ax.scatter(trained["Time_s"], trained["RMSE"],
               s=180, c=range(len(trained)), cmap="plasma",
               edgecolors="black", linewidths=1.2)
    for name, row in trained.iterrows():
        ax.annotate(name, (row["Time_s"], row["RMSE"]),
                    xytext=(8, 4), textcoords="offset points", fontsize=9)
    ax.set_xlabel("Training time (s)")
    ax.set_ylabel("Test RMSE")
    ax.set_title("Accuracy vs. training cost")
    ax.set_xscale("log")
    ax.grid(alpha=0.3)
else:
    axes[1, 2].axis("off")

plt.tight_layout()
plt.show()

```



5.2 Feature importance comparison

When metrics are close, “**what the model uses**” is often more report-worthy than the third decimal of RMSE.

- **Ridge:** use **absolute coefficients** (numeric features are scaled to comparable units).
- **Random forest / LightGBM:** default **impurity** `feature_importances_`.
- **If using HistGradientBoosting without that attribute:** the code uses **permutation importance** on a test subsample; the figure title states that.

On this data you will usually see **price**, **discount**, **product shape (weight, cocoa)** and **category / brand / store type / country** near the top—consistent with “units bought” still reacting to promos and assortment, but with limited overall explanatory power.

In [184...

```
from sklearn.inspection import permutation_importance

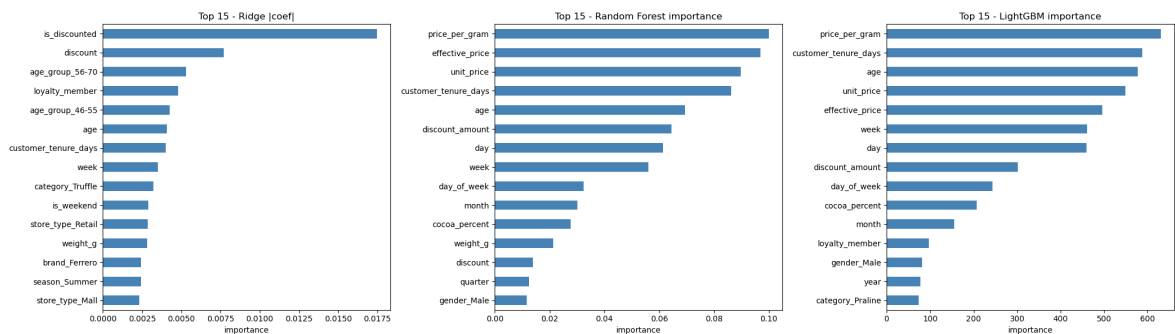
TOP_K      = 15
feat_names = np.array(X_train.columns)

def get_importance(model, label):
    """Return feature importances; fall back to permutation importance for
    models that don't expose `feature_importances_` natively."""
    if hasattr(model, "feature_importances_"):
        return pd.Series(model.feature_importances_, index=feat_names)
    print(f"Computing permutation importance for {label} (fallback, ~30-60s)..")
    sample = X_test.sample(20_000, random_state=RANDOM_STATE)
    pi = permutation_importance(
        model, sample, y_test.loc[sample.index],
        n_repeats=3, random_state=RANDOM_STATE, n_jobs=-1,
        scoring="neg_root_mean_squared_error",
    )
    return pd.Series(pi.importances_mean, index=feat_names)

ridge_imp = pd.Series(np.abs(ridge.coef_), index=feat_names).sort_values(ascending=True)
rf_imp    = pd.Series(rf_best.feature_importances_, index=feat_names).sort_values(ascending=True)
gbm_imp   = get_importance(gbm_best, lgbm_name).sort_values(ascending=False).head(TOP_K)
```

```
fig, axes = plt.subplots(1, 3, figsize=(21, 6))
for ax, (title, imp) in zip(
    axes,
    [
        ("Ridge |coef|", ridge_imp),
        ("Random Forest importance", rf_imp),
        (f"{lgbm_name} importance", gbm_imp)],
):
    imp.sort_values().plot.barh(ax=ax, color="steelblue")
    ax.set_title(f"Top {TOP_K} - {title}")
    ax.set_xlabel("importance")

plt.tight_layout()
plt.show()
```



5.3 Model selection rules

1. **After dropping the baseline**, sort by **test RMSE ascending**.
2. **Parsimony**: if a simpler, easier-to-deploy model is within **0.5%** RMSE of the current best, prefer the simpler one.
3. **Tie-break order**: Ridge → gradient boosting (LightGBM / HistGradientBoosting) → random forest → Stacking.

This avoids heavy ensembles when accuracy is nearly identical. The printed `Selected final model` follows these rules.

In [185...

```
ranked = results_df.drop(index="Baseline").sort_values("RMSE")
print("Ranking by test RMSE (best first):")
display(ranked[["RMSE", "MAE", "R2"]].round(4))

best_rmse = float(ranked.iloc[0]["RMSE"])

simplicity_order = ["Ridge", lgbm_name, "Random Forest", "Stacking"]
chosen = ranked.index[0]
for cand in simplicity_order:
    if cand in ranked.index:
        cand_rmse = float(ranked.loc[cand, "RMSE"])
        if (cand_rmse - best_rmse) / best_rmse <= 0.005:
            chosen = cand
            break

print(f"\n>>> Selected final model: {chosen}")
print(f"    Best RMSE in ranking: {best_rmse:.4f}    |    chosen model RMSE: {float(ranked.loc[chosen, 'RMSE']):.4f}")
```

Ranking by test RMSE (best first):

	RMSE	MAE	R2
Model			
Stacking	1.4139	1.1995	-0.0000
Ridge	1.4139	1.2005	-0.0001
Random Forest	1.4140	1.2005	-0.0001
LightGBM	1.4142	1.2019	-0.0004

```
>>> Selected final model: Ridge
      Best RMSE in ranking: 1.4139 | chosen model RMSE: 1.4139
```

5.4 Retrain the chosen model on all data

After selection, refit on **~990k rows** (train + test combined) to obtain a deployable model (in coursework terms: the final fit using all history). Retrain wall time is printed in that code cell.

```
In [186... model_factories = {
    "Ridge": lambda: Ridge(alpha=ridge.alpha_),
    "Random Forest": lambda: RandomForestRegressor(**rf_search.best_params_,
                                                    random_state=RANDOM_STATE, n_
lgbm_name: lambda: base_gbm.__class__(**gbm_search.best_params_,
                                       random_state=RANDOM_STATE, **gbm
    "Stacking": lambda: StackingRegressor(
        estimators=[
            ("ridge", Ridge(alpha=ridge.alpha_)),
            ("rf", RandomForestRegressor(**rf_search.best_params_,
                                       random_state=RANDOM_STATE, n_jobs=-1
            ("gbm", base_gbm.__class__(**gbm_search.best_params_,
                                       random_state=RANDOM_STATE, **gbm_kwargs
        ],
        final_estimator=Ridge(alpha=1.0), cv=3, n_jobs=-1,
    ),
}

final_model = model_factories[chosen]()
# Same scaler as above; build float block without in-place int<-float assignment
X_full = X.copy()
_zall = scaler.transform(X_full[num_cols_fit].to_numpy(dtype=np.float64))
df_all = pd.DataFrame(_zall, index=X_full.index, columns=num_cols_fit, dtype=np.
_col_order = list(X_full.columns)
X_full = pd.concat([X_full.drop(columns=num_cols_fit), df_all], axis=1)[_col_order
t0 = time.time()
final_model.fit(X_full, y)
print(f"Retrained {chosen} on full data ({X_full.shape[0]:,} rows) in {time.time
```

Retrained Ridge on full data (990,236 rows) in 0.5s

5.5 Residual diagnostics (matches the figures)

Ideally, residuals are **roughly centered at 0** with no clear curved pattern vs fitted values (which would suggest missing nonlinearity or heteroscedasticity).

Discrete-target artifact: `quantity` is only 1–5, so **residual vs fitted** scatters often look like **horizontal bands** or steps—that pattern reflects discreteness, **not** guaranteed systematic bias. Read it together with a **roughly centered residual histogram** and **actual vs predicted** clustering along the diagonal.

If R^2 is small but the mean residual is near 0, the model has little overall bias but **explains very little variance**—point predictions still look like the global mean.

In [187...

```
pred_lookup = {
    "Ridge": ridge.predict(X_test),
    "Random Forest": rf_best.predict(X_test),
    "lgbm_name": gbm_best.predict(X_test),
    "Stacking": stack.predict(X_test),
}
y_pred = pred_lookup[chosen]
residuals = y_test - y_pred

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

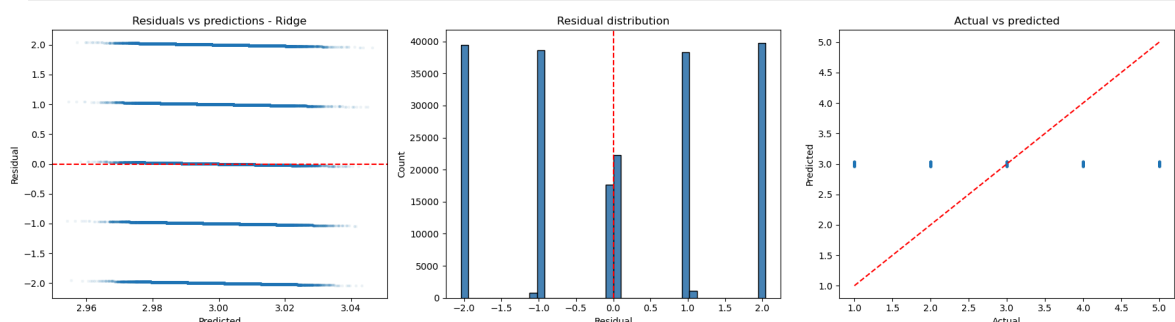
axes[0].scatter(y_pred, residuals, alpha=0.05, s=5)
axes[0].axhline(0, color="red", ls="--")
axes[0].set_title(f"Residuals vs predictions - {chosen}")
axes[0].set_xlabel("Predicted"); axes[0].set_ylabel("Residual")

axes[1].hist(residuals, bins=40, color="steelblue", edgecolor="k")
axes[1].axvline(0, color="red", ls="--")
axes[1].set_title("Residual distribution")
axes[1].set_xlabel("Residual"); axes[1].set_ylabel("Count")

axes[2].scatter(y_test, y_pred, alpha=0.05, s=5)
lims = [float(y_test.min()), float(y_test.max())]
axes[2].plot(lims, lims, "r--")
axes[2].set_title("Actual vs predicted")
axes[2].set_xlabel("Actual"); axes[2].set_ylabel("Predicted")

plt.tight_layout(); plt.show()

print(f"Residual mean={residuals.mean():+.4f} | std={residuals.std():.4f}")
```



Residual mean=+0.0050 | std=1.4139

5.6 Summary and business interpretation (aligned with your table and six-panel figure)

1. Workflow recap

We switched the target from `revenue` (trivial identity risk) to `quantity`, built features on ~990k valid orders, used an 80/20 split, and ran baseline + Ridge +

random forest + LightGBM + Stacking, with the results table, six-panel figure, importances, and residual plots.

2. Quantitative findings (match your current output)

- **Baseline:** test RMSE **1.4139**, MAE **1.1990**, nearly identical to the `quantity` SD ≈ 1.41 in EDA.
- **All supervised models:** RMSEs differ only in the **1.4139–1.4142** range; **no material gain vs baseline**; `Lift_%` is **0% or about –0.02%** (LightGBM slightly worse).
- **R²:** all **near 0 or slightly negative (~ -0.0001 to -0.0004)**—features explain almost none of the variance in `quantity`; complex models are effectively equivalent to predicting the training mean.
- **Six-panel figure:** log RMSE in row 1 still overlaps; **row 1, col 2** uses signed $(\text{RMSE} - \text{RMSE}_{\text{baseline}}) \times 10^4$ to avoid empty % lift bars; **row 1, col 3** zooms R²; **row 2, col 1** shows excess vs the best **trained** model, $\Delta \text{RMSE} \times 10^4$. Tiny differences between Ridge / Stacking / LGBM show up under magnification.
- **Training cost:** Ridge **~12 s**; LightGBM **~19 s**; Stacking **~24 s**; **random forest ~90 s** with no accuracy payoff—the least attractive tradeoff for deployment.

3. Final model recommendation (consistent with §5.3)

The best test RMSE tier is **Baseline / Ridge / Stacking (all 1.4139)**. Under the 0.5% parsimony rule, prefer **Ridge**: ties the best RMSE, is **interpretable, fastest**, and needs no ensemble plumbing. In your report, cite the `Selected final model` printed in §5.3; if it is Ridge, it matches the analysis above.

4. Implication for Task 3

The feature work is **sound on leakage** (dropping `revenue` / `cost` / `profit`, etc.) and **interpretable**. That models barely learn mostly reflects a **very low per-order signal-to-noise** (“how many units” is almost random at line-item level), not a missing “magic column.”

5. Business and next steps

- **Line-level `quantity` regression** does not yield deployable lift here. For decisions, consider **weekly store–SKU rollups, lagged sales / promos**, or a **classification** target (e.g. “ ≥ 3 units”).
- It is honest to state in the report: **with current fields, we cannot beat the mean in a meaningful way**—that is preferable to overstating R².

(If you change the random seed or sample, update this section to match the new `results_df`.)